

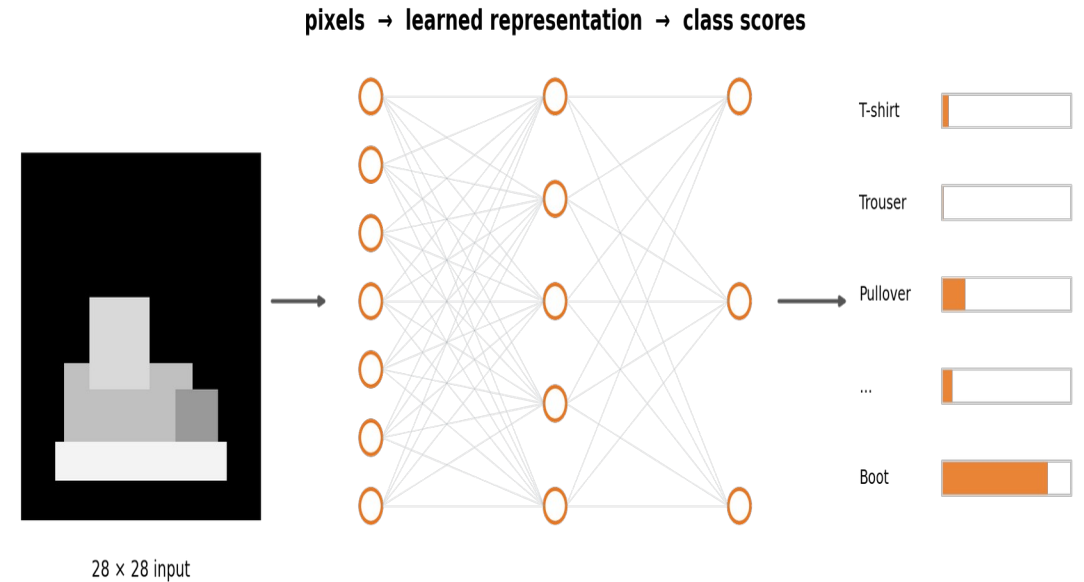
POSTGRADUATE PROGRAMME IN EMBEDDED SYSTEMS

Neural Networks

Data, Neurons & Edge ML

THEORY SESSION 04

Data, weighted decisions and embedded constraints



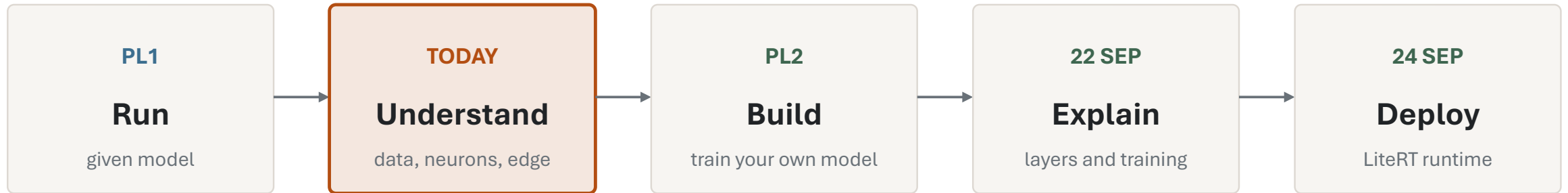
17 September 2026 | Tiago Fonseca | ISEP

What should you be able to do after today?

- Identify inputs, targets and the roles of training, validation and test data.
- Explain how weights, bias and activation determine a neuron's output.
- Use AND, OR and XOR to explain why hidden layers matter.
- Distinguish parameter storage from the working memory used by activations.
- Relate predictive quality to compute, memory and energy requirements.

These concepts prepare the model comparisons and deployment work in PL2.

Where this lecture sits in the course



PL1 introduced inference. Today we examine the data, the neuron and the cost of a model.

Today's route

01 Data before networks

inputs, labels, splits

02 Neurons & logic

weights, bias, activations

03 Edge perspective

storage, RAM, compute

01

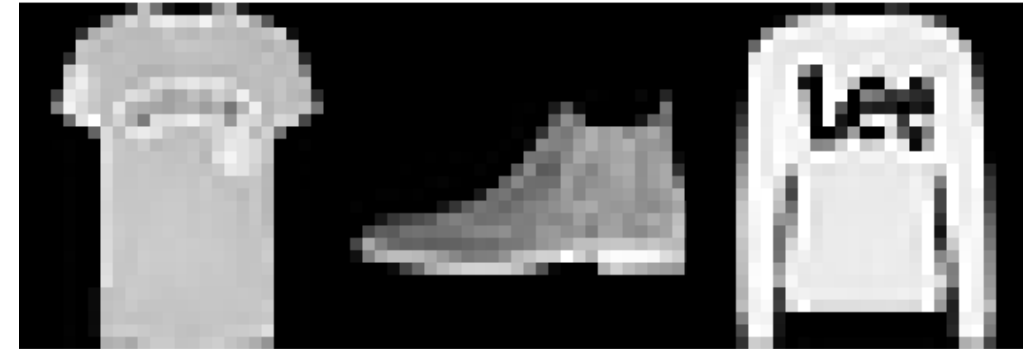
Data before networks

Before discussing neurons, we need to be precise about what the learning problem actually is.

Fashion-MNIST as our running example

The same dataset will connect theory, PL1 and PL2.

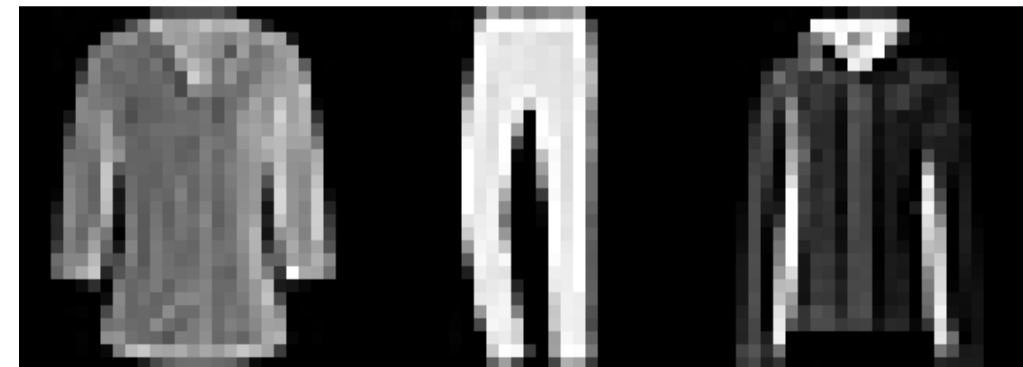
- 70,000 grayscale images of clothing items, each 28×28 pixels.
- 10 classes: T-shirt/top, trouser, pullover, dress, coat, sandal, shirt, sneaker, bag, ankle boot.
- Simple enough to inspect; non-trivial enough to expose ambiguity and model mistakes.
- We will keep returning to the same examples while adding neural-network concepts.



T-shirt/top

Ankle boot

Pullover

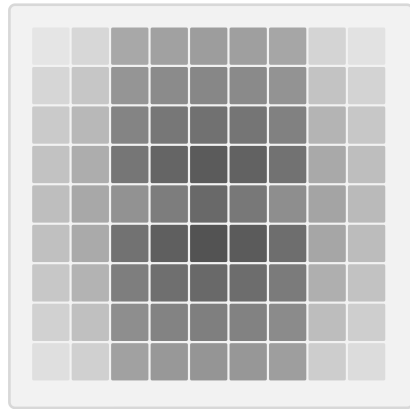


Shirt

Trouser

Coat

One supervised-learning sample = input + target



$x = 784$ pixel values

Input x

A numeric representation of one observation. Here: a 28×28 grayscale image.

Target y

The correct class label supplied during supervised training.

Learning problem

Learn a function $f(x; \theta)$ that maps image pixels to useful class scores. θ denotes the model parameters that training will adjust.

Four words you need before we train anything

If these four are clear, the rest of the lecture becomes much easier.

1

Sample

One example given to the model.

Example: One Fashion-MNIST image

2

Feature

One measured input value used to describe a sample.

Example: One pixel intensity; one sensor reading

3

Label / target

The correct answer supplied for training.

Example: "Ankle boot" → class 9

4

Class

One possible category the model may predict.

Example: 10 clothing categories

For Fashion-MNIST: one sample = one 28×28 image; the 784 pixel values are the input features; the label is one integer from 0 to 9.

What does the model actually receive?

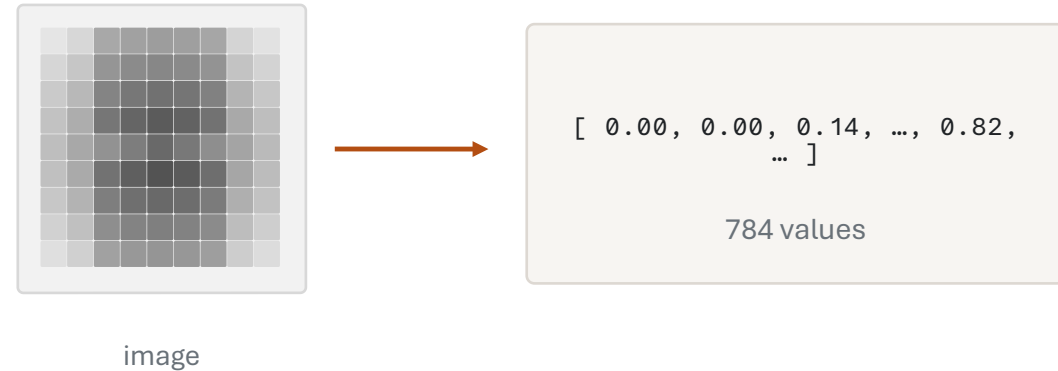
Not “a shoe” — a tensor of numbers.

An image is converted into numerical pixel intensities.

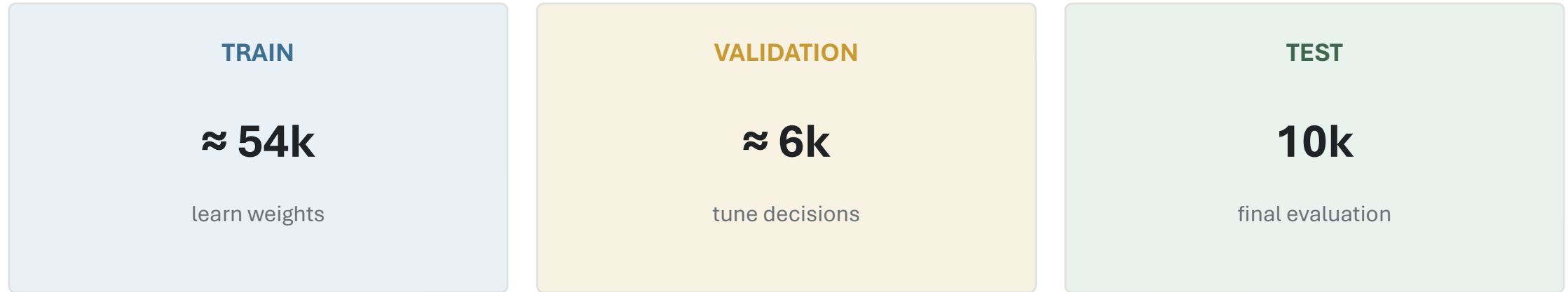
A tensor is simply a multidimensional array of numbers — a convenient container for model inputs and intermediate values.

For our MLP, 28×28 pixels are flattened into 784 input values.

Semantic meaning is not encoded explicitly; it must emerge from learned patterns.



Train, validation and test are different jobs



If the test set influences training decisions, it stops being an honest test.

Typical PL2 split: use the training set for learning, reserve part of it for validation, and keep the official test set untouched until evaluation.

Why must the test set stay unseen?

Because evaluation should approximate future unseen data.

- Training optimizes parameters on examples the model is allowed to learn from.
- Validation helps us choose design decisions without touching the final test set.
- The test set estimates generalization after those choices are fixed.
- Repeatedly “peeking” at test results leaks information and makes performance look better than it really is.

Data leakage

Information used outside its allowed role: for example, fitting preprocessing on held-out data or choosing a model using test results.

Engineering analogy

Testing a device with the exact faults you used to tune it is not an independent reliability test.

Normalization changes numerical scale — not semantic content

Fashion-MNIST is stored as uint8 values from 0 to 255.



uint8 pixel values

0 ... 255

$$x_{\text{normalized}} = x / 255.0$$

float32 model input

Same ordering and visual information, but values now lie in [0, 1].

The important rule: training and inference must use the same input convention.

Classification is a score-producing problem

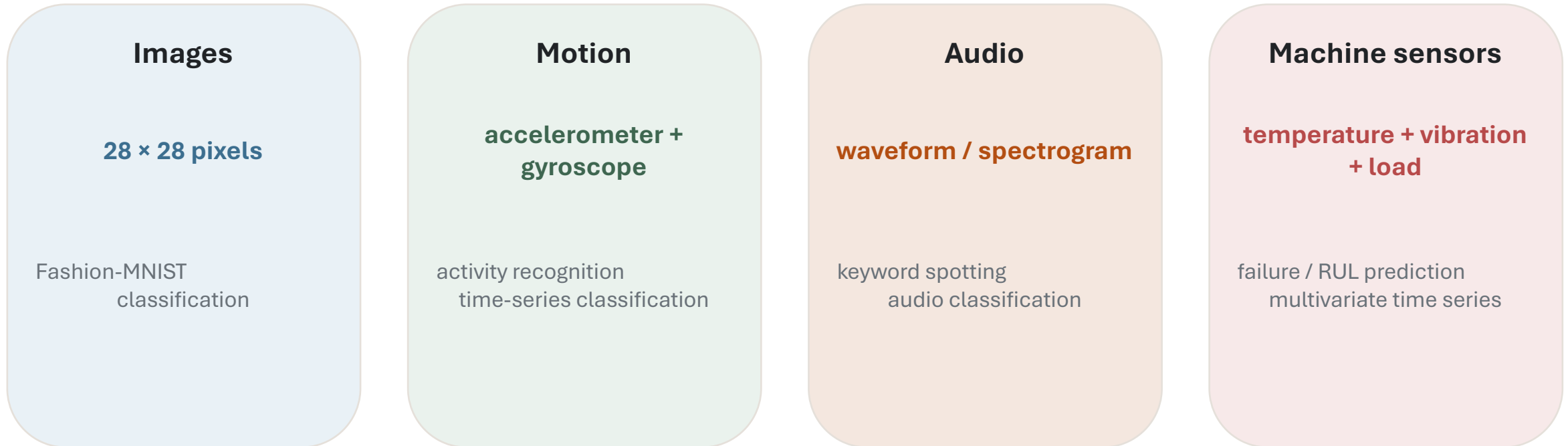
The network will not output the word “coat” directly.

- Input: numerical representation of one sample.
- Model: transforms the input through layers of learned parameters.
- Output: one score per class.
- Decision: usually choose the class with the highest output score.



Datasets can look very different

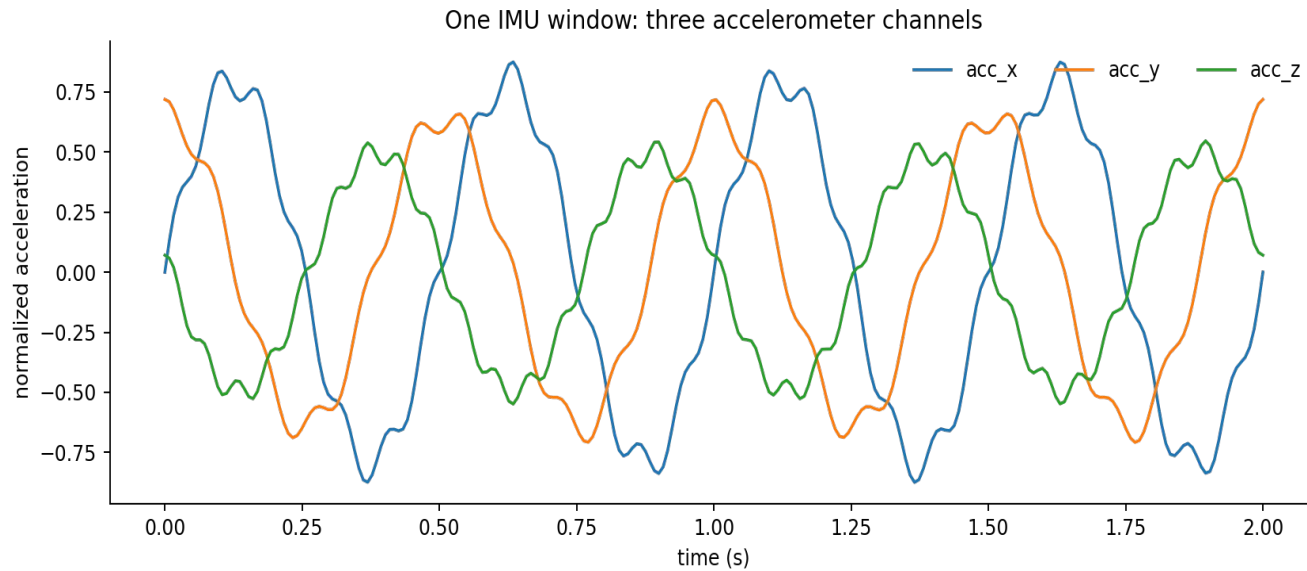
The learning vocabulary stays the same: samples, inputs, targets and an evaluation protocol.



Different data types change the model architecture and preprocessing — not the need for a clean target and honest evaluation.

Example dataset: human activity from motion sensors

Not an image: each sample is a short multichannel time window.



Sample

A window of accelerometer / gyroscope values collected over time.

Target

Activity label such as walking, sitting or standing.

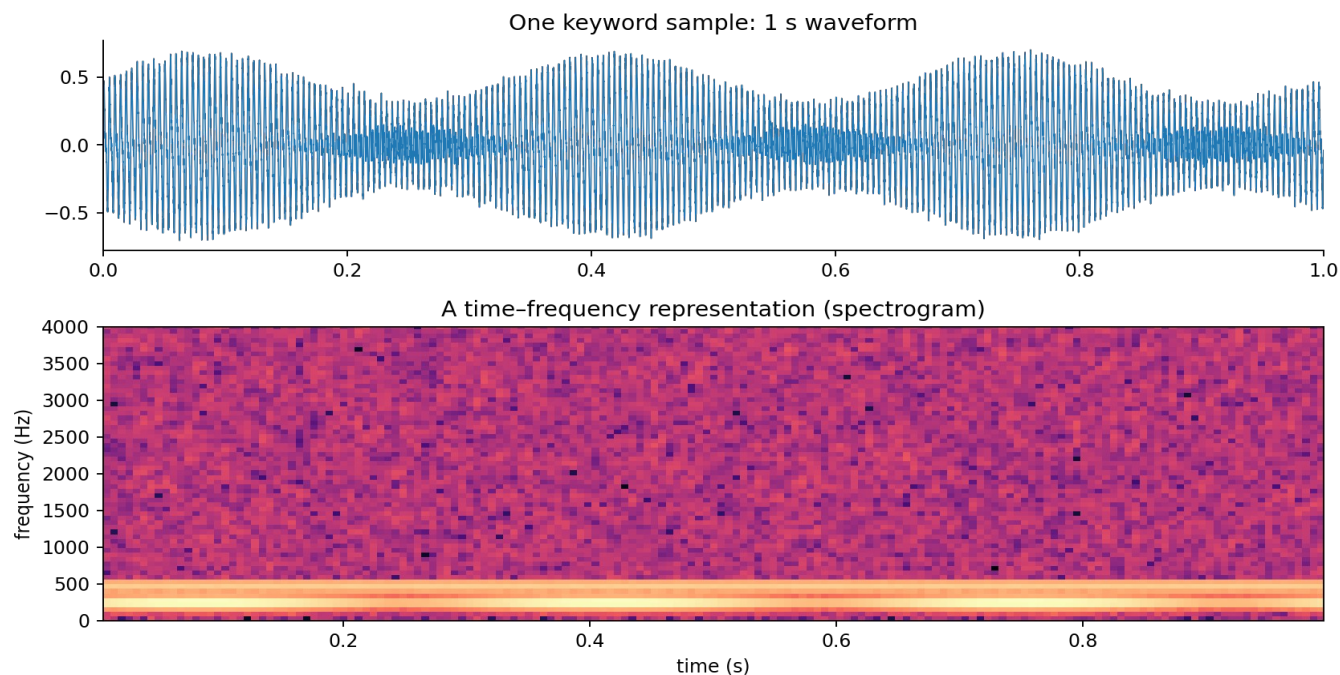
Embedded angle

Wearables and phones can classify locally with low latency and no raw-data upload.

What changes vs Fashion-MNIST? Sequence order matters, sensor orientation matters, and different users can move differently.

Example dataset: keyword spotting from audio

The input is a signal; the target can still be one class among many.



Sample

A short waveform — often transformed into a spectrogram or learned representation.

Target

A keyword such as “yes”, “no”, “up”, “down” or a wake word.

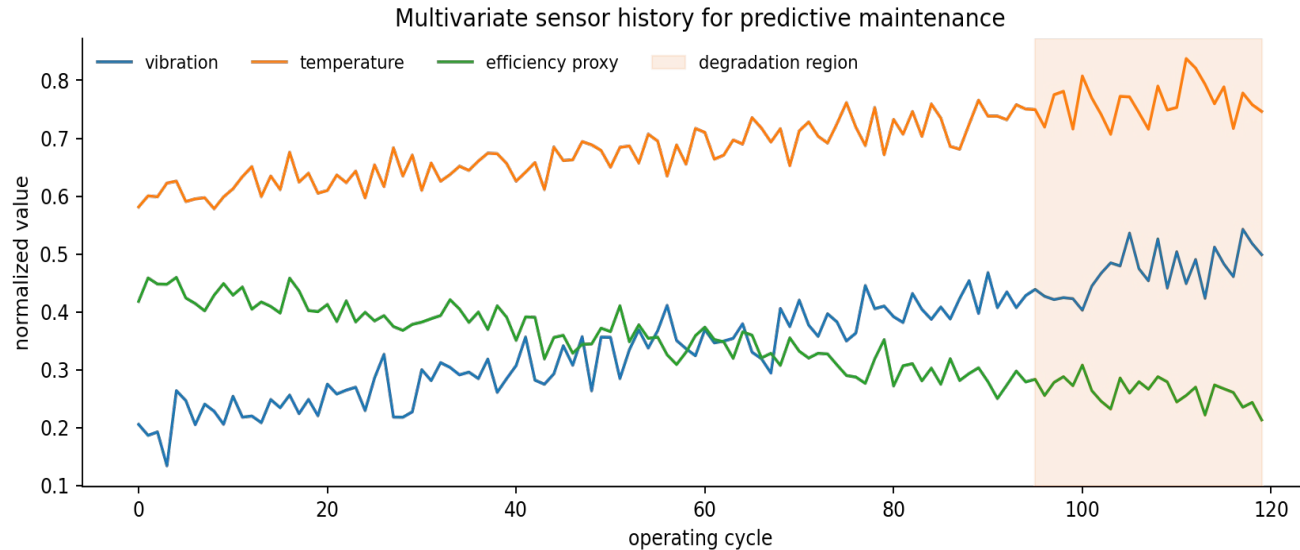
Embedded angle

Always-on local inference avoids continuously streaming microphone audio.

New challenge: background noise, microphones, accents and speaking rate create variation that the model must generalize across.

Example dataset: predictive maintenance from machine sensors

Some ML tasks predict a condition or remaining lifetime rather than an image class.



Sample

One sensor vector or a short history of vibration, temperature, pressure and load.

Target

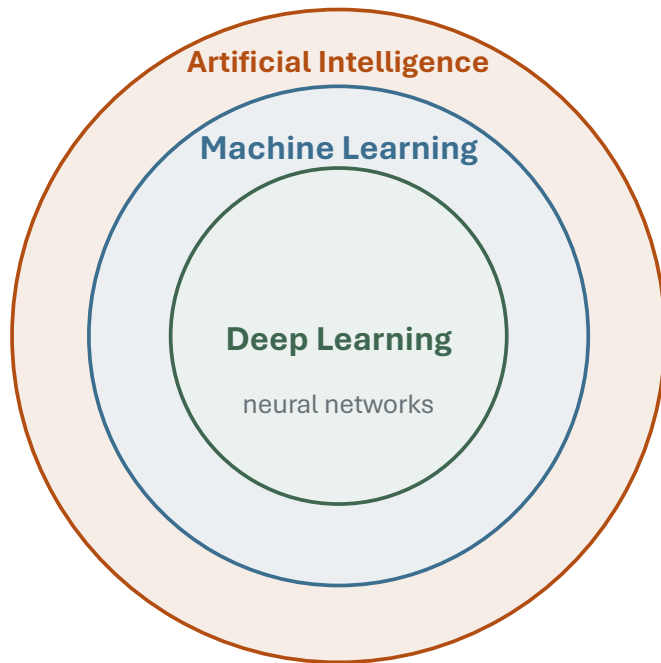
Failure / healthy state, anomaly score, or remaining useful life.

Embedded angle

Local monitoring can react near the equipment and reduce bandwidth.

New challenge: failures are rare, operating conditions drift, and the most important errors may be false negatives rather than average accuracy.

AI, Machine Learning and Deep Learning



Artificial Intelligence

Broad goal: systems that perform tasks requiring intelligent behavior.

Machine Learning

A subset of AI where behavior is learned from data rather than fully hand-coded.

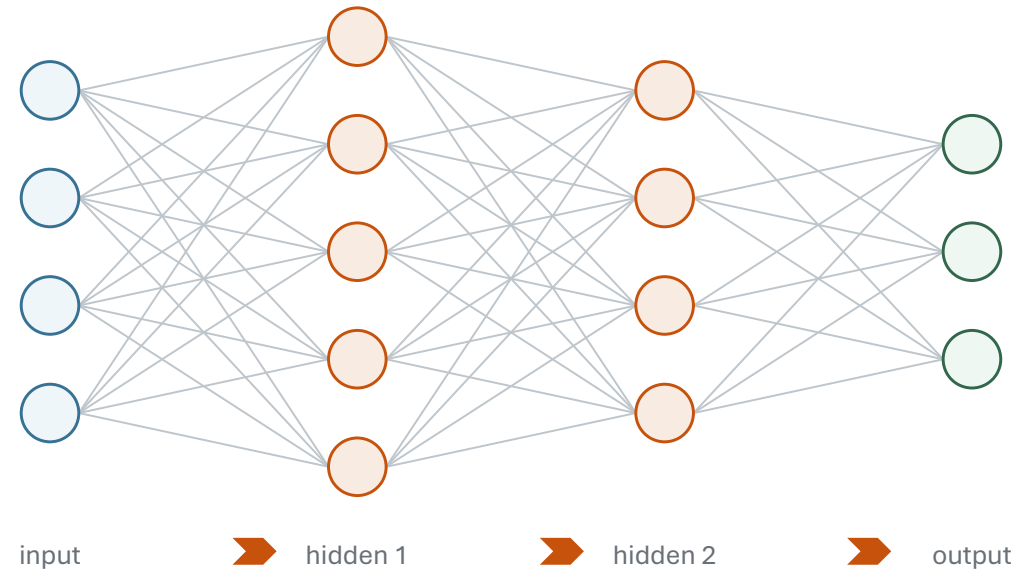
Deep Learning

A subset of ML based on multi-layer neural networks that learn representations from data.

Why “deep”?

Depth means composing multiple learned transformations.

- A single linear layer can only express limited decision boundaries.
- Hidden layers allow the network to transform the representation before the final decision.
- More layers can model richer structure — but also cost more compute, memory and data.
- For Edge ML, “deep” does not automatically mean “large”.



Composition

Each hidden layer transforms the representation produced by the previous layer.

Before we build a network...

If the input is just 784 numbers, how can a network ever discover “shoe”, “coat” or “bag”?

The answer will emerge as we move from one neuron to hidden representations.

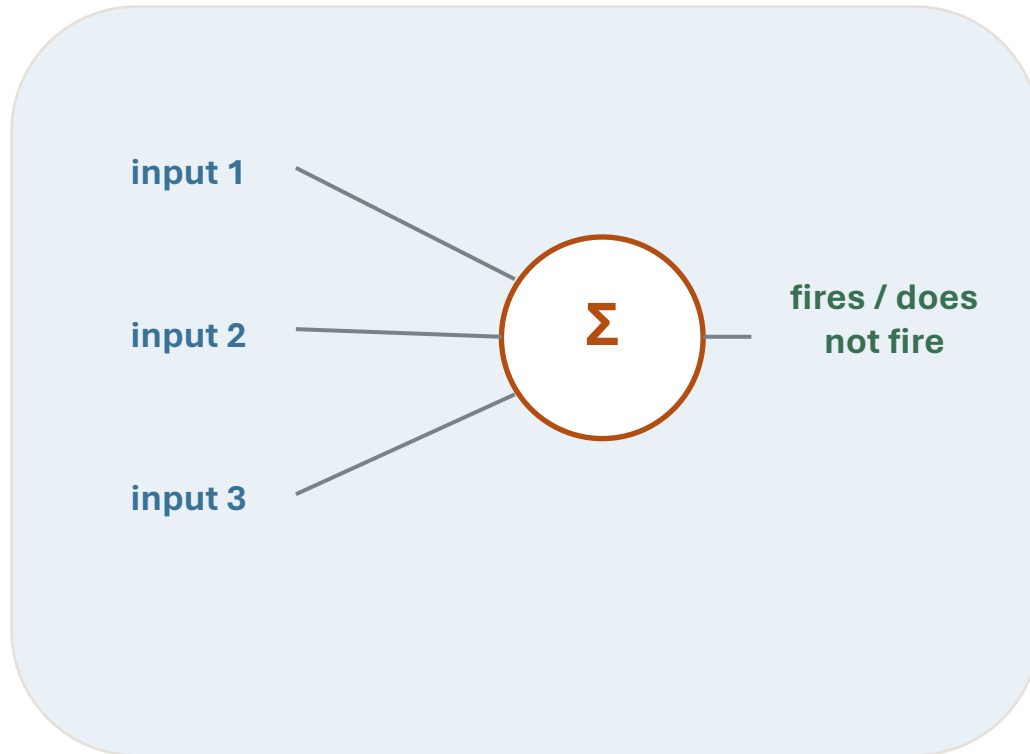
02

Neurons & logic

Start with one weighted decision. Then see why hidden layers become necessary.

1943 — From biological inspiration to logical neurons

McCulloch and Pitts proposed a mathematical abstraction of a neuron as an all-or-none threshold unit.



What was new?

A network of simple threshold units could represent logical computations.

What was missing?

No modern gradient-based learning pipeline; the emphasis was logical computation and neural abstraction.

Why remember it?

The weighted-sum + threshold idea is the ancestor of the neuron we use today.

Historical reference: McCulloch & Pitts, "A Logical Calculus of the Ideas Immanent in Nervous Activity" (1943).

1957–1958 — Rosenblatt’s perceptron learns from examples

The perceptron turned the threshold neuron into a trainable linear classifier.

The key step

Instead of hand-setting every weight, update the weights when the prediction is wrong.

Learning becomes an iterative process driven by examples.

1957 Perceptron algorithm

A learnable linear decision rule.



1958

Mark I Perceptron

A physical visual-pattern classifier implemented at the Cornell Aeronautical Laboratory.

The core limitation remained: one perceptron produces a linear decision boundary.

Historical reference: Rosenblatt, “The Perceptron” (1958).

From a single perceptron to multilayer networks

The limitation was not “neural networks do not work”; it was that one linear layer cannot represent every useful function.

1969 — limits made explicit

Single-layer perceptrons cannot represent XOR and other non-linearly separable problems. The mathematical limitation became impossible to ignore.

1980s — multilayer training

Backpropagation provided an efficient way to compute gradients through multiple layers, making practical multilayer training far more accessible.

2000s+ — “deep” learning

More data, faster hardware and improved optimization allowed much deeper networks to become practical at scale.

Historical references: Minsky & Papert (1969); Rumelhart, Hinton & Williams (1986).

2012 and beyond — deep learning becomes practical at scale

The boom came from several ingredients arriving together: data, compute, architectures and optimization.

2012

AlexNet + ImageNet

A GPU-trained convolutional network dramatically improved large-scale image classification and made deep learning impossible to ignore.

2014–2016

Deeper, easier-to-train networks

VGG, GoogLeNet and especially ResNet showed that much deeper models could be trained effectively and could keep improving accuracy.

2017+

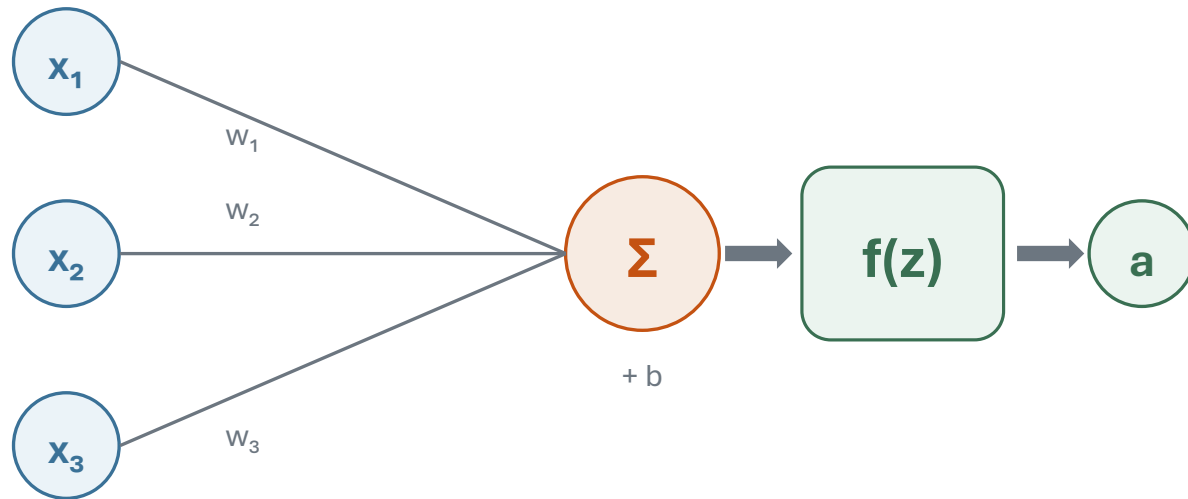
Attention, scale and new hardware

Transformers broadened the impact beyond vision, while GPUs, accelerators and better software made training and inference increasingly practical.

Key idea: 2012 was not the invention of neural networks. It was the moment when old ideas, large datasets and modern compute converged at scale.

References: Krizhevsky, Sutskever & Hinton (2012); He et al. (2016); Vaswani et al. (2017).

A neuron turns weighted evidence into an activation



Weighted sum

$$z = w_1x_1 + w_2x_2 + w_3x_3 + b$$

Activation

$$a = f(z)$$

Weights determine how strongly each input contributes. The bias adds a learnable offset. The activation decides what value leaves the neuron.

Three ingredients of one neuron

Think of the neuron as a tiny scoring rule.

1

Weight

How strongly should this input matter?

positive → supports the score
negative → opposes the score

2

Bias

A learnable offset added after the weighted sum.

moves the decision threshold
without changing the inputs

3

Activation

A function applied to the score before it leaves the neuron.

ReLU: keep positive values
and replace negative values by 0

Training will learn the weights and biases. We choose the activation function as part of the architecture.

Worked example: one neuron, real numbers

Inputs

$$x_1 = 0.8$$
$$x_2 = 0.3$$

Parameters

$$w_1 = 1.2$$
$$w_2 = -0.5$$
$$b = 0.1$$



$$z = 1.2 \cdot 0.8 - 0.5 \cdot 0.3$$
$$+ 0.1 = 0.91$$

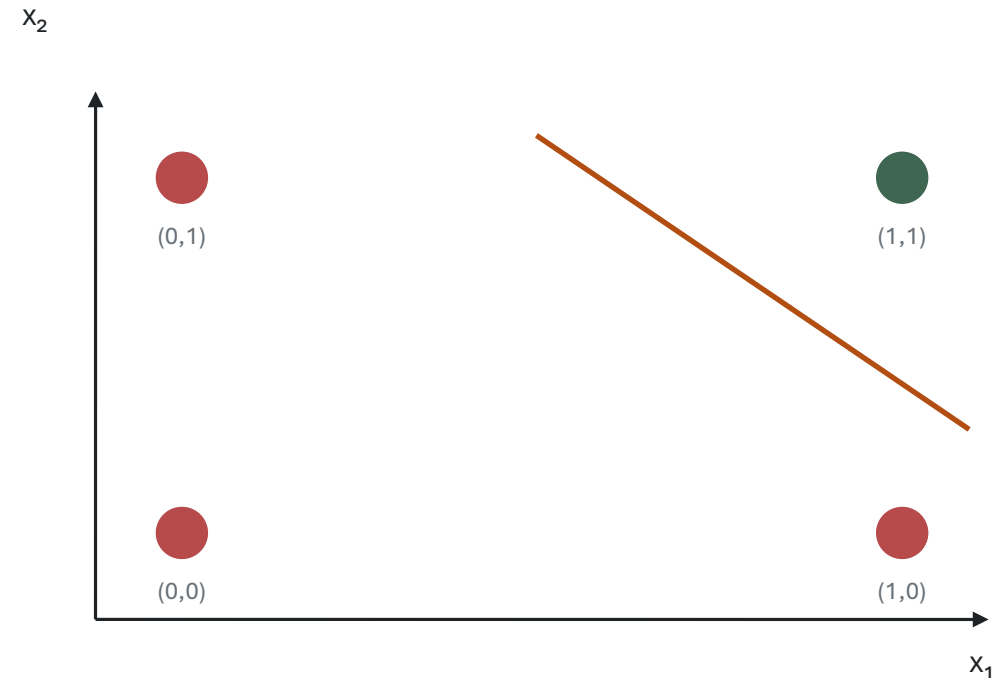
$$\text{ReLU}(z) = \max(0,$$
$$0.91) = 0.91$$

The inputs are numbers. Weights say how strongly each input matters; the bias shifts the score. ReLU then decides what value leaves the neuron.

A neuron defines a decision boundary

With two inputs, the pre-activation $z = w_1x_1 + w_2x_2 + b$ is linear.

- The weights determine the orientation of the boundary.
- Changing the bias shifts the boundary.
- A threshold activation can turn the linear score into a binary decision.
- This is enough for some logical functions — but not all.



Perceptron: the historical threshold neuron

**output = 1 if $w^T x$
 + b > 0**

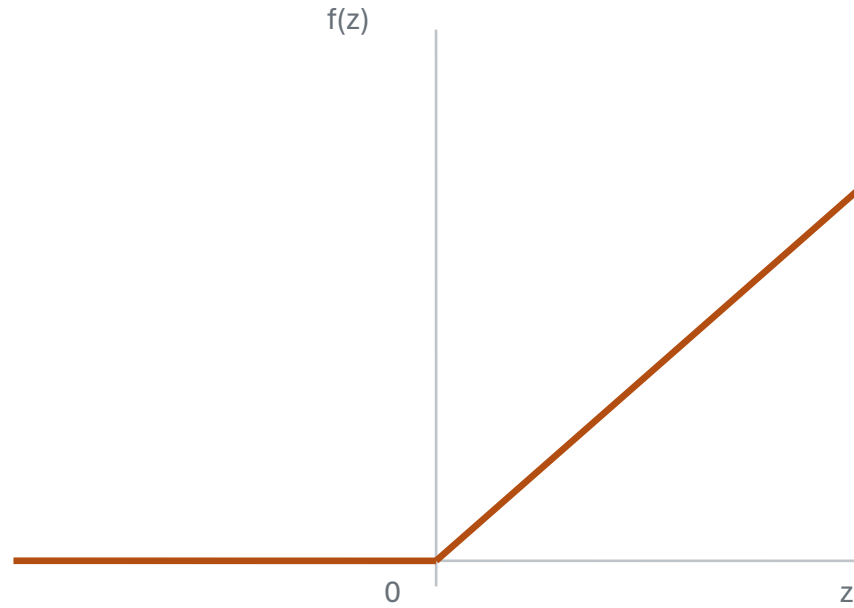
output = 0 otherwise

Why it matters

The perceptron makes the core idea visible: inputs → weighted sum → decision. Modern neural networks keep weighted sums, use activations suitable for gradient-based training, and connect many units.

ReLU: simple and effective

$$f(z) = \max(0, z)$$



If $z < 0$

output = 0

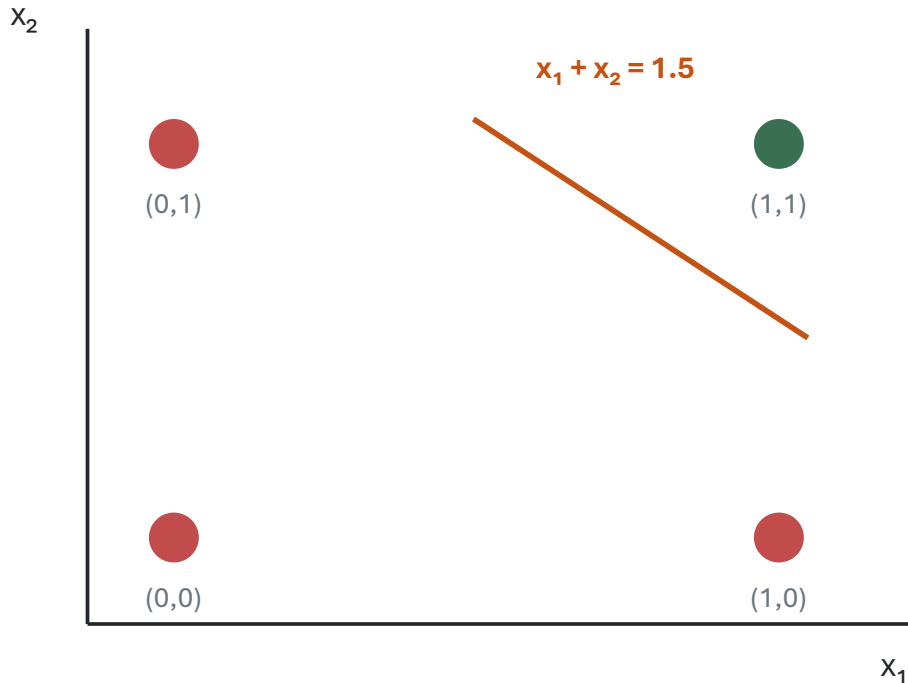
If $z > 0$

output = z

Edge note

Cheap operation: compare with zero and pass the positive value through.

AND can be implemented by one neuron



One possible threshold rule

$$z = x_1 + x_2 - 1.5$$

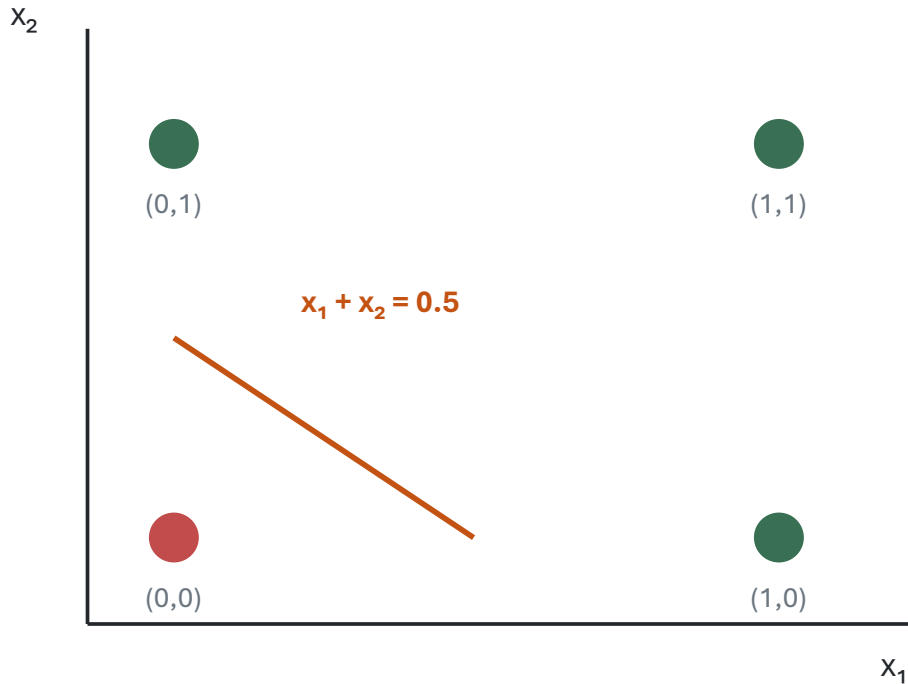
output = 1 if $z > 0$

Check the four inputs

(0,0): $-1.5 \rightarrow 0$
(1,0): $-0.5 \rightarrow 0$
(0,1): $-0.5 \rightarrow 0$
(1,1): $+0.5 \rightarrow 1$

Only the point (1,1) lies on the positive side.

OR can also be implemented by one neuron



One possible threshold rule

$$z = x_1 + x_2 - 0.5$$

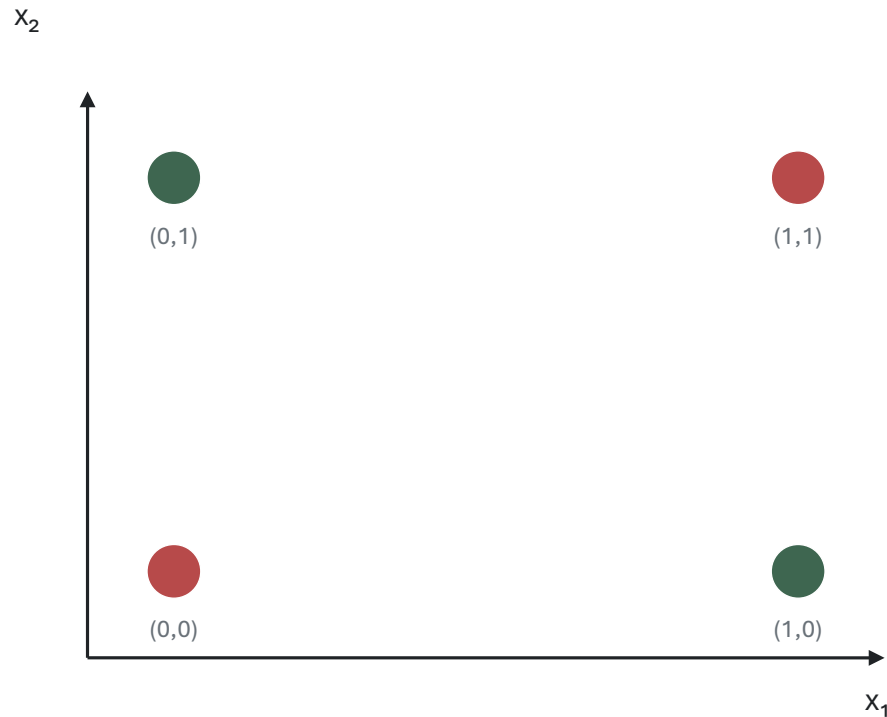
output = 1 if $z > 0$

Check the four inputs

(0,0): $-0.5 \rightarrow 0$
(1,0): $+0.5 \rightarrow 1$
(0,1): $+0.5 \rightarrow 1$
(1,1): $+1.5 \rightarrow 1$

Only the point (0,0) lies on the negative side.

XOR breaks the single-neuron story



No single straight line separates the classes.

XOR truth table

$0 \text{ XOR } 0 = 0$	$1 \text{ XOR } 0 = 1$
$0 \text{ XOR } 1 = 1$	$1 \text{ XOR } 1 = 0$

Problem

The positive points sit on opposite corners. No single straight boundary can separate them from the negatives.

Why the activation function matters

Stacking linear layers without a nonlinearity is still just one linear transformation.

- Dense $\rightarrow$ Dense with no activation can be algebraically collapsed into one Dense layer.
- A nonlinear activation prevents that collapse.
- This is what allows hidden layers to create more expressive decision boundaries.
- ReLU is a common default for small hidden layers.

$$W_2 (W_1 x + b_1) + b_2 = W' x + b'$$

Still linear

$$W_2 \text{ReLU}(W_1 x + b_1) + b_2$$

Nonlinear composition

Common activation functions

Different roles, different behaviors.

ReLU

hidden layers

$\max(0, z)$

Sigmoid

binary outputs / gates

0..1

Tanh

centered gating

-1..1

Softmax

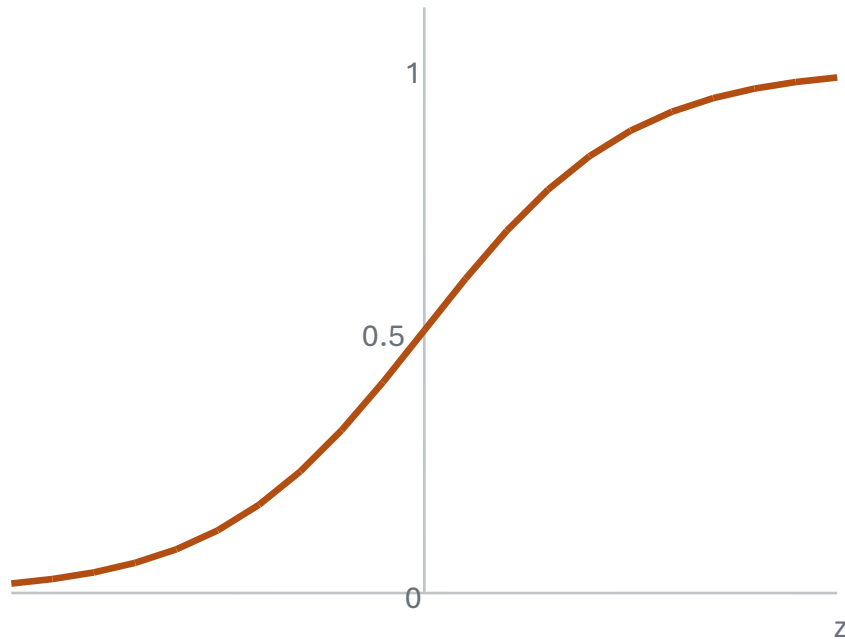
multi-class output

scores sum to 1

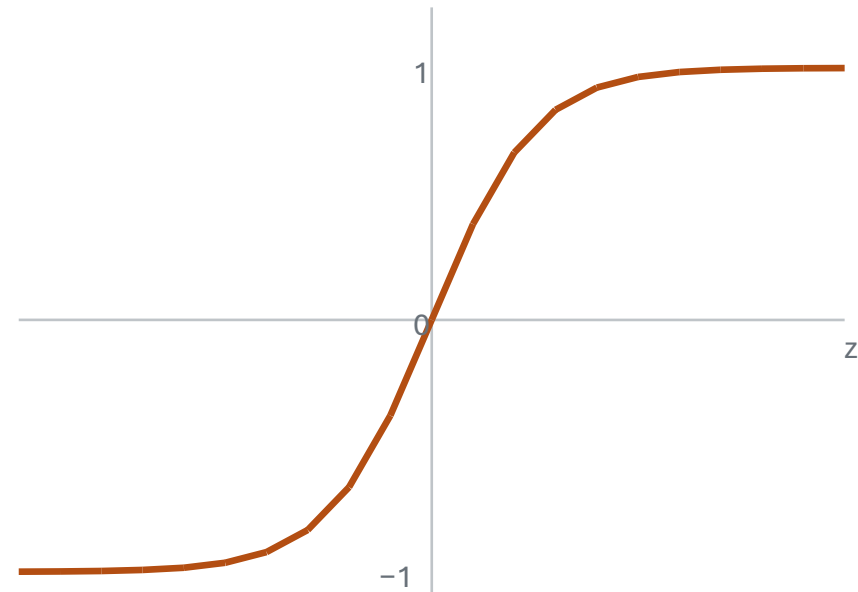
For our PL2 MLP: ReLU in the hidden layer, Softmax at the 10-class output.

Sigmoid and tanh: useful, but not our hidden-layer default

Sigmoid



Tanh



Both saturate at the extremes, which can make gradients very small.

Softmax turns logits into comparable class scores

Three-class example for arithmetic; Fashion-MNIST uses 10 outputs.

Before Softmax — logits

Pullover 2.7
Coat 1.3
Shirt 0.8



After Softmax — normalized scores

Pullover 0.716
Coat 0.177
Shirt 0.107
 $\Sigma = 1$

A high Softmax score is not automatically a well-calibrated probability of being correct.

Checkpoint: what happens if we remove ReLU?

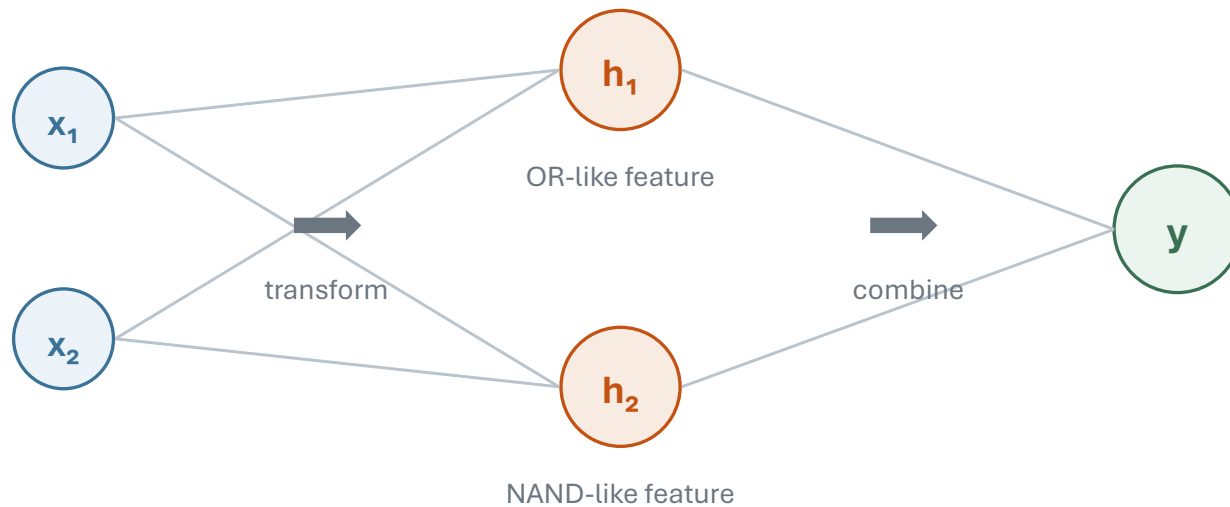
Think first — then discuss with the person next to you.

If we stack Dense layers but put no nonlinear activation between them, do we really gain a more expressive model?

Hint: composing linear transformations still gives another linear transformation. ReLU is what lets the hidden layer bend the representation.

A hidden layer can solve XOR

The network first transforms the representation, then separates it.



Why this works

The hidden layer builds two intermediate conditions. In that transformed space, the final decision becomes linearly separable.

Main idea

Hidden layers do not magically solve XOR. They change the representation so that a later linear decision becomes possible.

XOR in equations: two hidden neurons are enough

One constructive solution uses an OR-like neuron, a NAND-like neuron and a final AND-like neuron.

Hidden layer

$$h_1 = \text{step}(x_1 + x_2 - 0.5) \rightarrow \text{OR}$$

$$h_2 = \text{step}(-x_1 - x_2 + 1.5) \rightarrow \text{NAND}$$

Output neuron

$$y = \text{step}(h_1 + h_2 - 1.5) \rightarrow \text{AND}$$

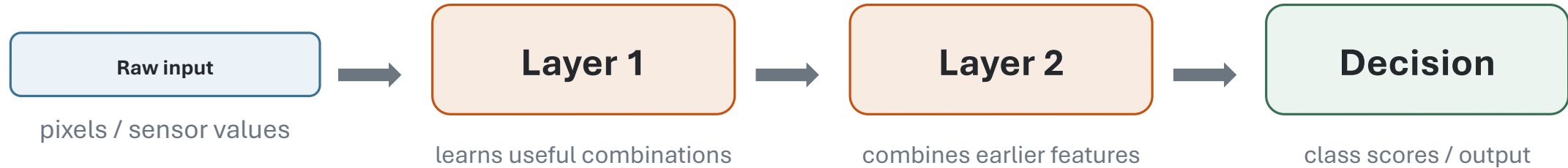
Verify all four cases

$x_1 \ x_2$	h_1	h_2	y
0 0	0	1	0
1 0	1	1	1
0 1	1	1	1
1 1	1	0	0

The hidden layer creates a new representation $[h_1, h_2]$. In that representation, the final XOR decision is again linearly separable.

Hidden layers learn representations

This is the central idea behind deep learning.



Representation learning

Training adjusts the transformations jointly. Hidden units are not manually assigned meanings; they become useful because they help reduce the final loss.

Fashion-MNIST intuition

Pixels may be recombined into responses to edges, strokes, openings or other patterns. Later layers can combine those responses into evidence for classes such as shoe, coat or shirt.

Think: what would a hidden unit mean in Fashion-MNIST?

Could one hidden unit “detect sleeves”, another “detect footwear”, another “detect vertical edges”?

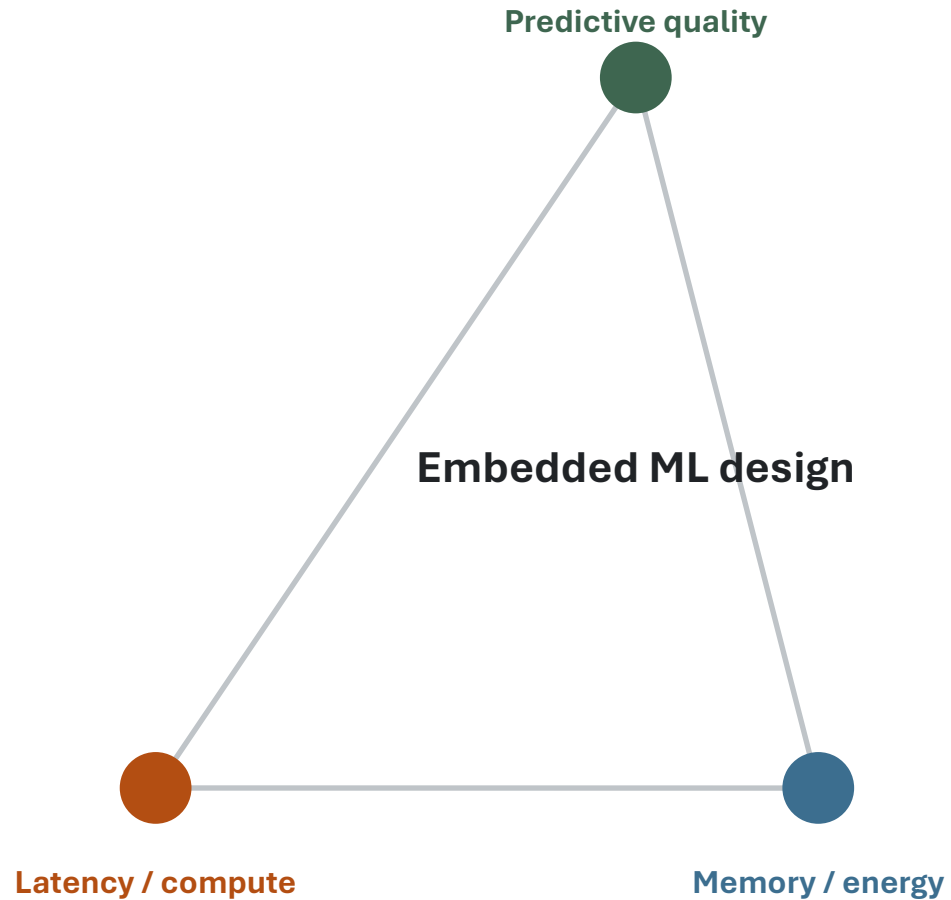
Possibly — but learned representations need not align cleanly with human labels.

03

Edge perspective

A neural network is not only a predictor. It is also a computational workload.

Neural-network design becomes an engineering trade-off on edge devices



Quality

Does the model meet the predictive requirement?

Latency / compute

Can the hardware execute it fast enough?

Memory / energy

Does it fit the device budget sustainably?

Parameters consume persistent storage and memory

Parameter count is the first cost you can estimate before deployment.

- Float32 parameter = 4 bytes.
- Baseline: 7,850 parameters → 31.4 kB of parameter data.
- PL2 MLP: 50,890 parameters → 203.56 kB of parameter data.
- Real model files also include structure and metadata.

$$\begin{aligned} 50,890 \times 4 \text{ B} &= 203,560 \text{ B} \\ &= 203.56 \text{ kB} \approx 198.79 \text{ KiB} \end{aligned}$$

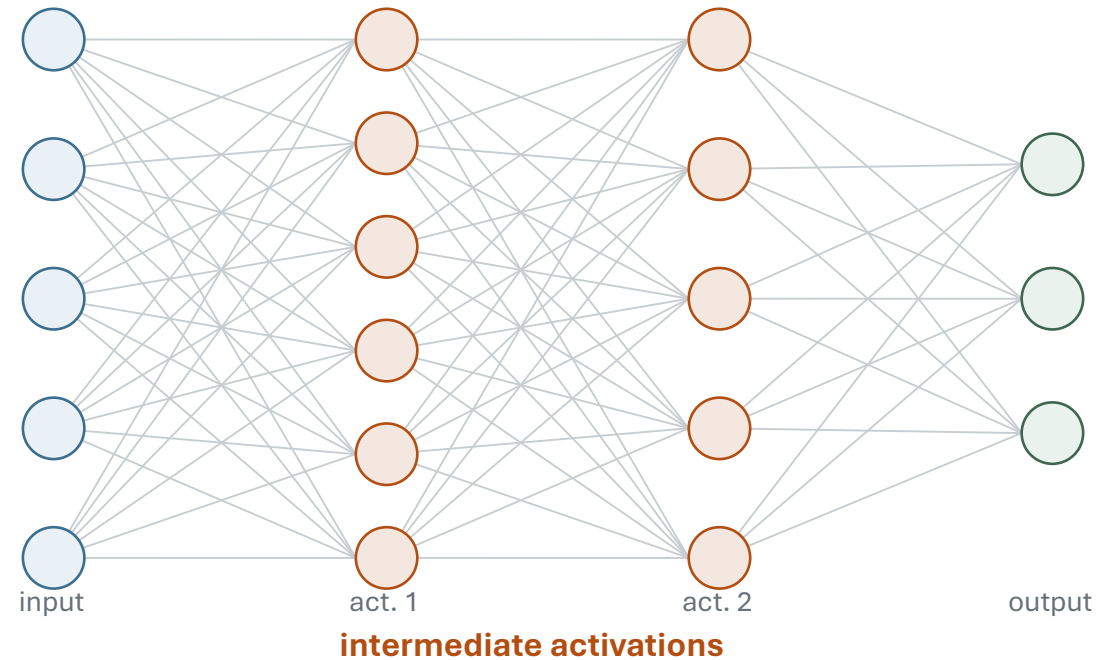
Later in PL3

Changing numeric representation can reduce this cost dramatically.

Activations consume working memory during inference

Weights are not the only memory cost.

- Each layer produces intermediate tensors.
- Some runtimes can reuse buffers; others need multiple tensors alive simultaneously.
- Larger layers / feature maps increase working-memory pressure.
- A model can fit on disk but still be uncomfortable in RAM.



Every connection implies arithmetic

Dense layers translate directly into multiply-accumulate work.

- Dense(784 → 64): ~50k weight multiplications per sample.
- Dense(64 → 10): another ~640.
- More units / layers generally add arithmetic; measure latency and energy on the target.
- Hardware libraries can optimize these operations — but the work does not disappear.

$$784 \times 64$$

≈ 50,176 multiply terms

$$64 \times 10$$

≈ 640 multiply terms

Baseline vs MLP: what changed?

Same dataset. Same output classes. Different capacity and cost.

Baseline

7,850 params

linear decision

lower cost

lower capacity

MLP

50,890 params

nonlinear hidden features

higher cost

higher capacity

“Better model” depends on the application requirement — not one metric in isolation.

Every architecture choice becomes a hardware cost

The network design you choose in software becomes memory, arithmetic and energy on the device.

More hidden units / layers

→ **More parameters**

→ More stored weights and activations

Larger input

→ **More values enter the network**

→ More arithmetic per inference

Higher inference rate

→ **Same model, called more often**

→ Higher sustained CPU load and energy

This is why “more accurate” and “better for an embedded product” are not always the same statement.

Small-model thinking: decide the requirement before growing the network

Start with a small model and add complexity only when measured application requirements justify it.

1

Define

What accuracy / recall /
latency is actually
needed?

2

Start simple

Use a baseline that
exposes the real value of
extra capacity.

3

Add capacity deliberately

Change one design choice
and measure what you
gained and what it cost.

4

Stop when justified

Extra complexity needs
evidence, not intuition.

PL2 will let you compare exactly this: a linear baseline versus a small MLP using the same data and the same task.

Embedded example: wearable gesture recognition

Small temporal model, local inference, battery constraints.



Wearable IMU

accelerometer / gyroscope → gesture class

Why edge?

low latency, privacy, operation without network

Constraint

battery and RAM budget may matter more than a tiny accuracy gain

Possible model family

small MLP / 1D CNN / compact sequence model

Embedded example: keyword spotting

“Wake word” detection on a microphone stream.



Audio front-end

microphone → audio features → keyword / no keyword

Why edge?

instant response and no need to stream raw audio continuously

Constraint

model may run continuously — energy per inference accumulates

Possible model family

small CNN / depthwise-separable CNN

Embedded example: predictive maintenance from vibration

Sensor data → anomaly / fault class.



Machine sensor

vibration window → health / fault state

Why edge?

local decisions close to equipment; reduced bandwidth

Constraint

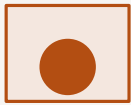
robustness to operating conditions may matter more than raw test accuracy

Possible model family

MLP / 1D CNN / autoencoder

Embedded example: local visual sorting

Camera → small vision network → actuator decision.



Vision station

camera crop → class scores → reject / route / count

Why edge?

low-latency control and reduced video transmission

Constraint

real images add lighting, scale, blur and background variability

Possible model family

small CNN — spatial structure matters

Edge ML design questions to ask before choosing a network

Input

What data arrives? At what rate?

Output

What decision is required?

Quality

What errors matter?

Latency

How fast must the answer arrive?

Memory

What must fit in RAM/storage?

Energy

How often will inference run?

Key takeaways

1

Data defines the task

2

Splits separate learning and evaluation

3

A neuron combines weighted inputs

4

Nonlinearity enables richer decisions

5

Parameters and activations cost memory

6

Requirements guide the model choice

Four questions you should now be able to answer

1. Why can a hidden layer solve XOR when one neuron cannot?
2. Why do we need separate training, validation and test data?
3. Why is model file size different from the RAM used during inference?
4. Why might a smaller model be the better embedded-system choice?

Closing thought

A useful embedded model solves the intended task within the device's compute, memory and energy budgets.

In PL2, compare model quality and deployment evidence. On 22 September, we examine training.

Further reading / reference points

Goodfellow, Bengio & Courville

Deep Learning — neural networks, optimization and generalization foundations.

Fashion-MNIST

Zalando Research — benchmark dataset used throughout the course.

François Chollet

Deep Learning with Python — practical Keras-oriented explanation.

Michael Nielsen

Neural Networks and Deep Learning — accessible introduction with intuitive explanations and worked examples.

Slides intentionally simplify mathematics to prioritize conceptual understanding and engineering reasoning.